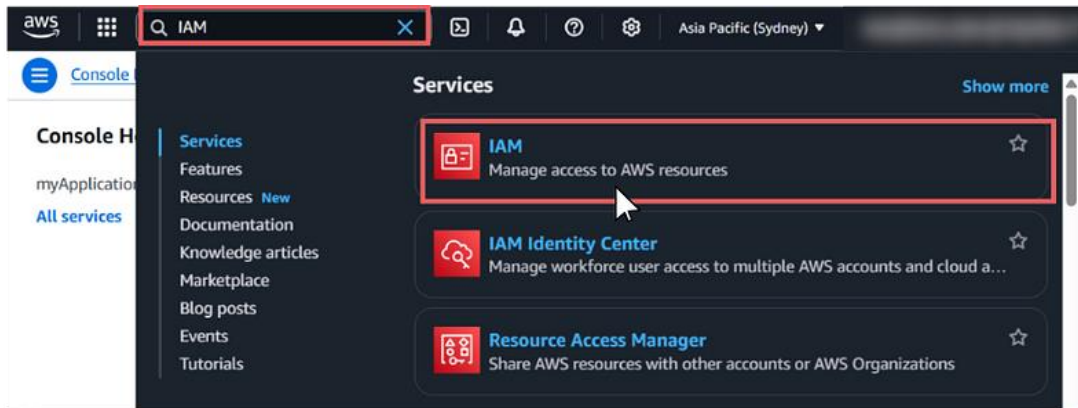


IAM USER

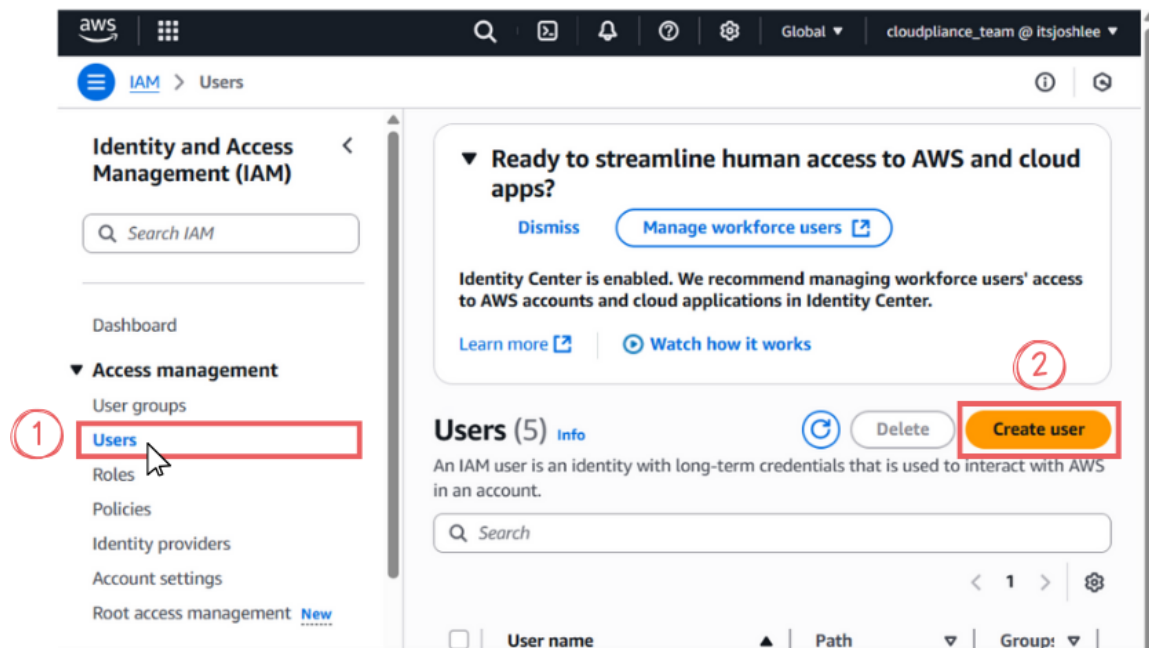
1. Go to IAM.

Press enter or click to view image in full size



2. Click "Users" and then "Create User."

Press enter or click to view image in full size



3. Enter user details and select access type.

On this screen, you'll define the new user's basic login details and how they'll access AWS services.

Here's what to do:

Step 1: Specify user details
 Step 2: Set permissions
 Step 3: Review and create
 Step 4: Retrieve password

Specify user details

User details

User name

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, -, @, _ (hyphen)

☒ Provide user access to the AWS Management Console - optional
 If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Are you providing console access to a person?

User type

☐ Specify a user in Identity Center - Recommended
 We recommend that you use Identity Center to provide console access to a person. With Identity Center, you can centrally manage user access to their AWS accounts and cloud applications.

☒ I want to create an IAM user
 We recommend that you create IAM users only if you need to enable programmatic access through access keys, service-specific credentials for AWS CodeCommit or Amazon Keyspaces, or a backup credential for emergency account access.

Console password

☐ Autogenerated password
 You can view the password after you create the user.

☒ Custom password
 Enter a custom password for the user:

Must be at least 8 characters long
 Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - { } | ' "

☐ Show password

☒ Users must create a new password at next sign-in - Recommended
 Users automatically get the [IAMUserChangePassword](#) policy to allow them to change their own password.

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user. [Learn more](#)

Cancel Next

- Enter a unique username for the user (e.g., dev_john, marketing_bot, etc.)
 - Enable AWS Management Console access if the user needs to log in through the AWS website. Under User type, choose I want to create an IAM user
 - Set a console password. Choose between *Autogenerated* or *Custom password*
 - (Recommended) Keep the “Users must create a new password at next sign-in” box checked for security. Then click Next.
4. Set permissions for the new user, you'll assign permissions by adding the user to an existing group.
- Here's what to do:

Step 1: Specify user details

Step 2: **Set permissions**

Step 3: Review and create

Step 4: Retrieve password

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

- ☒ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.
- ☐ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

User groups (1/1)

Search

☒	Group name	Users	Attached po...	Creat...
☒	admin	4	AdministratorA...	2024-...

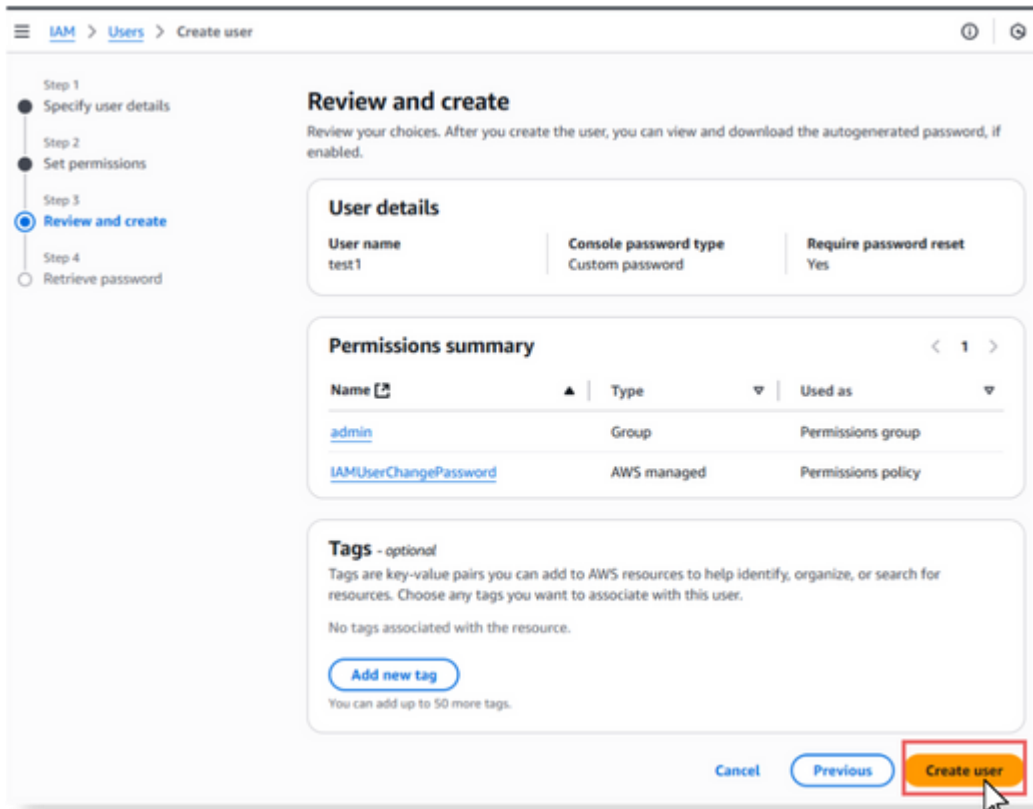
Set permissions boundary - optional

Set a permissions boundary to control the maximum permissions for this user. Use this advanced feature used to delegate permission management to others. [Learn more](#)

☐ Use a permissions boundary to control the maximum permissions
You can select one of the existing permissions policies to define the boundary.

Cancel Previous **Next**

- Under Permissions options, choose Add user to group
This is the recommended approach to manage user permissions by job role.
 - In the User groups list, check the box next to the group called admin
This will automatically grant the user all permissions associated with the *admin* group.
 - (Optional) You can create a new group if one doesn't exist by clicking Create group
 - Skip the Set permissions boundary section unless you need advanced restrictions and then click "Next"
5. Review everything and hit "Create user."

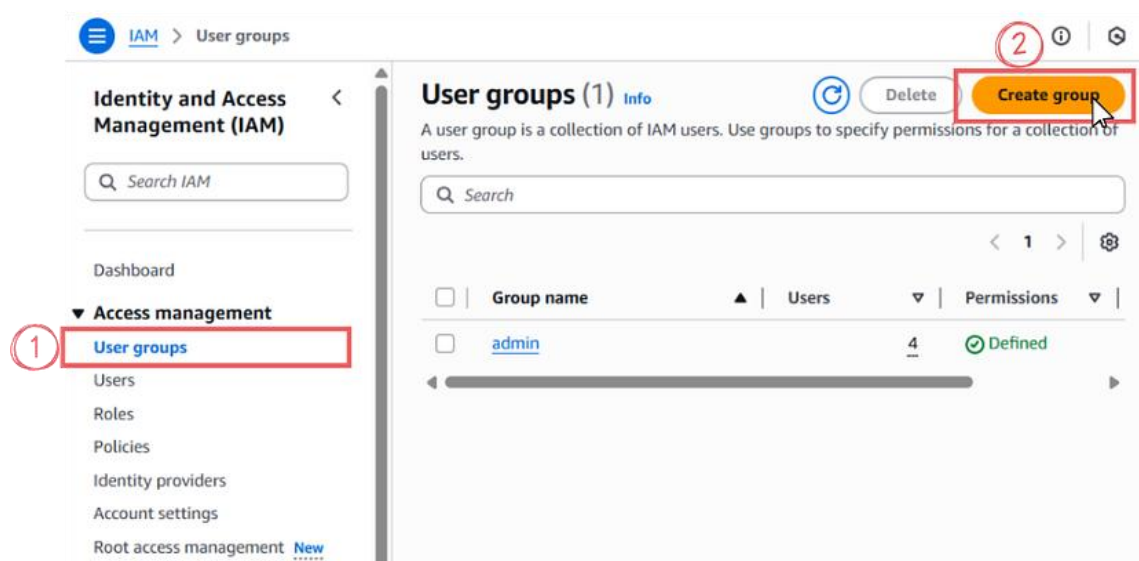


If you're giving them programmatic access, AWS gives you access keys. Make sure you save those somewhere safe — losing them is a pain.

Managing IAM Groups

Groups make life easier. Instead of setting permissions for every single user, you just create a group and drop users into it.

Press enter or click to view image in full size



When you set up a group:

Create user group

Name the group

User group name
Enter a meaningful name to identify this group.

Maximum 128 characters. Use alphanumeric and '+', '@', '-' characters.

Add users to the group - Optional (6) [Info](#)

An IAM user is an entity that you create in AWS to represent the person or application that uses it to interact with AWS.

Q Search

☐	User name	Groups	Last
☐	awscli	1	10/10/2019
☐	awscli	1	10/10/2019
☐	awscli	1	10/10/2019
☐	awscli	1	10/10/2019
☐	awscli	1	10/10/2019
☐	awscli	1	10/10/2019

Attach permissions policies - Optional (1061) [Info](#)

You can attach up to 10 policies to this user group. All the users in this group will have permissions that are defined in the selected policies.

Filter by Type

Q Search

Customer ... 3 matches

☐	Policy name	Type	Use...	Descr
☐	AWSLambdaBasicExec...	Customer man...	Permis...	-
☐	AWSLambdaBasicExec...	Customer man...	Permis...	-
☐	vsIMonkS3Bucket	Customer man...	None	-

[Cancel](#) [Create user group](#)

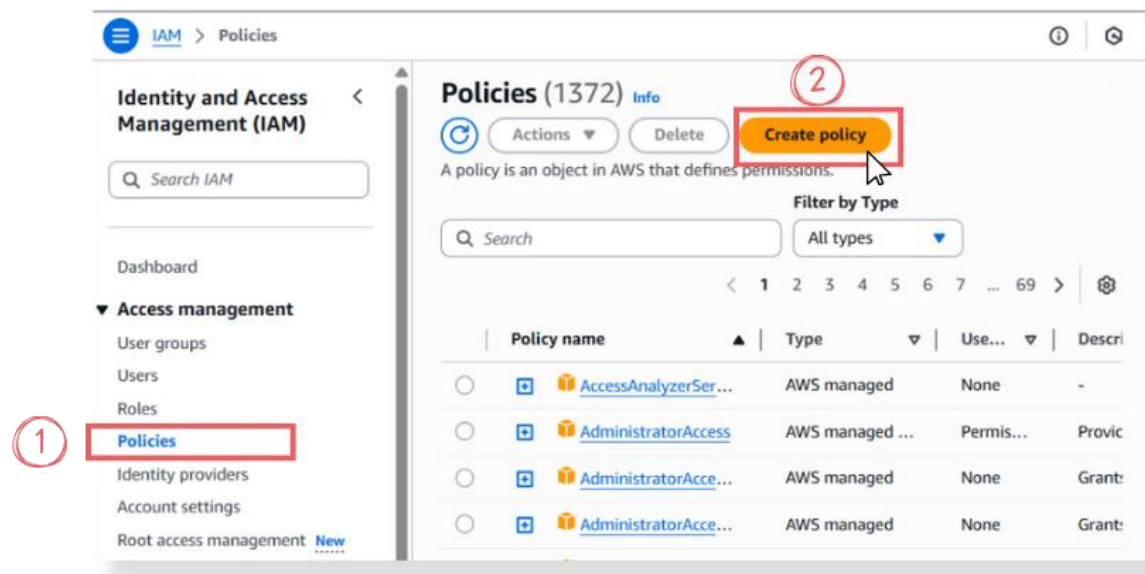
- Pick a name that actually means something (like “Developers” or “Admins”).
- Attach the policies that group needs.
- Add users, and they get those permissions automatically.

For example, if you’ve got a bunch of devs, just make a “Developers” group and give it the permissions they need. Done.

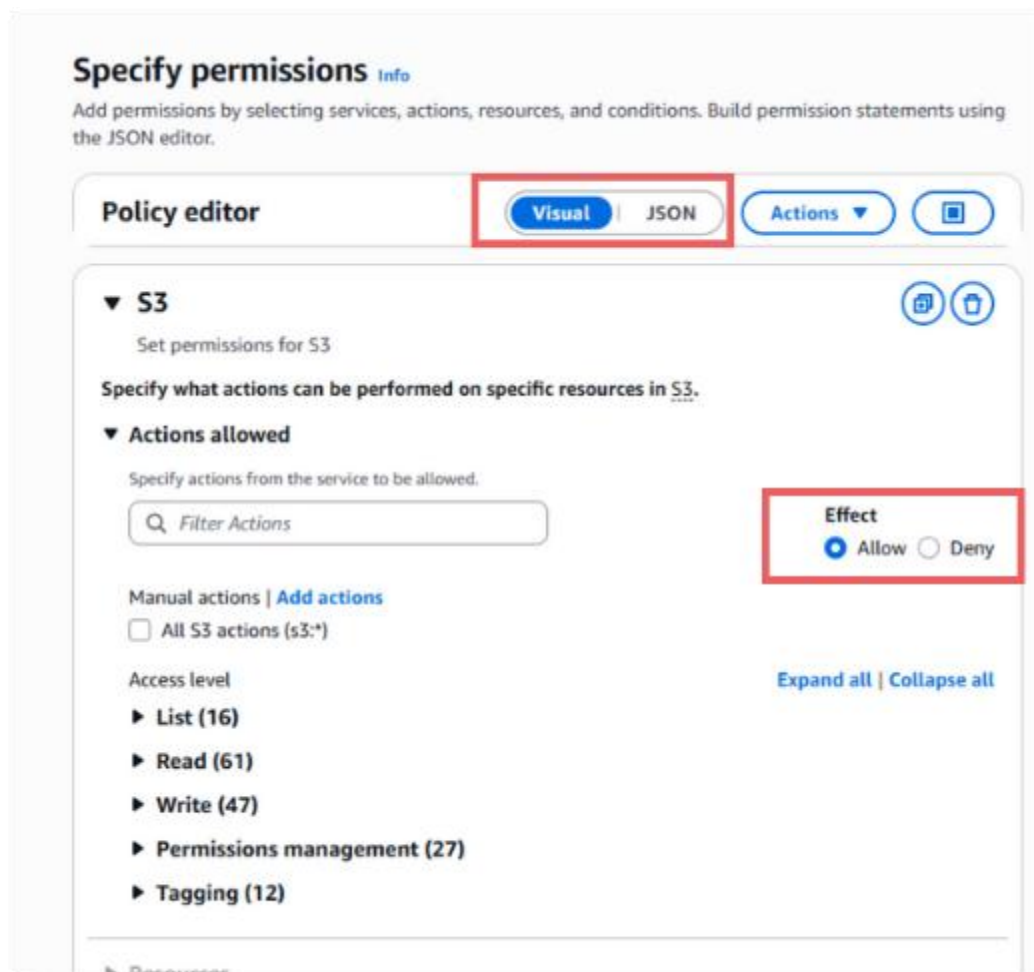
Configuring IAM Policies

Policies are just JSON files that lay out what someone can or can’t do.

Press enter or click to view image in full size



Some quick points about policies:



- They either Allow or Deny stuff.

- They're made up of statements that say what actions are allowed, and on which resources.
- You can get fancy and add conditions, like time of day or IP address.

AWS has a bunch of pre-made (managed) policies you can use. Or, if you like being in control, you can write your own custom ones.

Here's a super basic example that lets someone read from an S3 bucket:

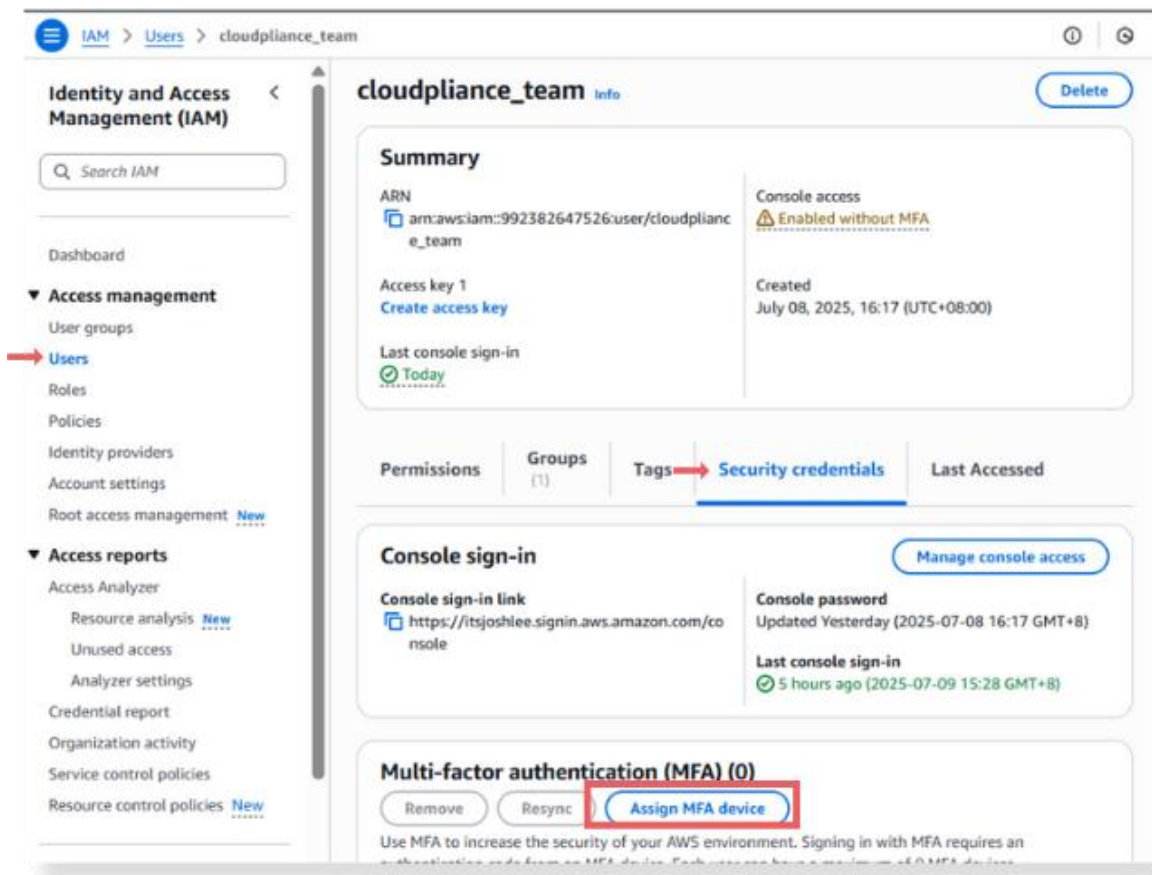
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": ["arn:aws:s3:::example-bucket", "arn:aws:s3:::example-bucket/*"]
    }
  ]
}
```

And yeah, always stick to the “least privilege” rule. Only give people what they actually need. No more, no less.

Securing and Managing Access

You want to know who's getting into your AWS stuff, and how. That means adding some extra security, keeping your permissions organized, and watching for anything weird going on.

Enabling Multi-Factor Authentication



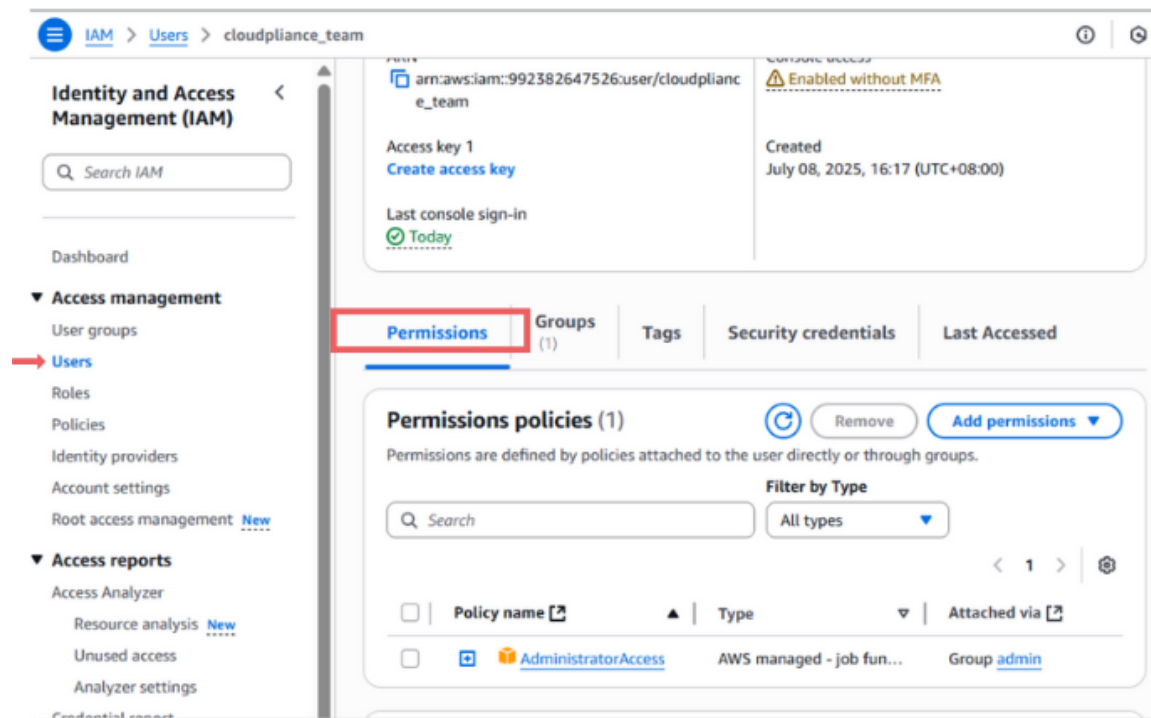
Multi-Factor Authentication (MFA) is just a fancy way of saying, “prove it’s really you.” After your password, you have to enter a code from your phone or a hardware device.

You can set up MFA for each user, or just for your main (root) account. It’s a lifesaver if someone steals a password. AWS works with apps like Google Authenticator, or you can use a physical thing like a YubiKey.

Honestly, turn on MFA for any account you care about. It’s quick to set up in the IAM console and you won’t have to mess with it much after that.

Implementing Role-Based Access Control

Press enter or click to view image in full size

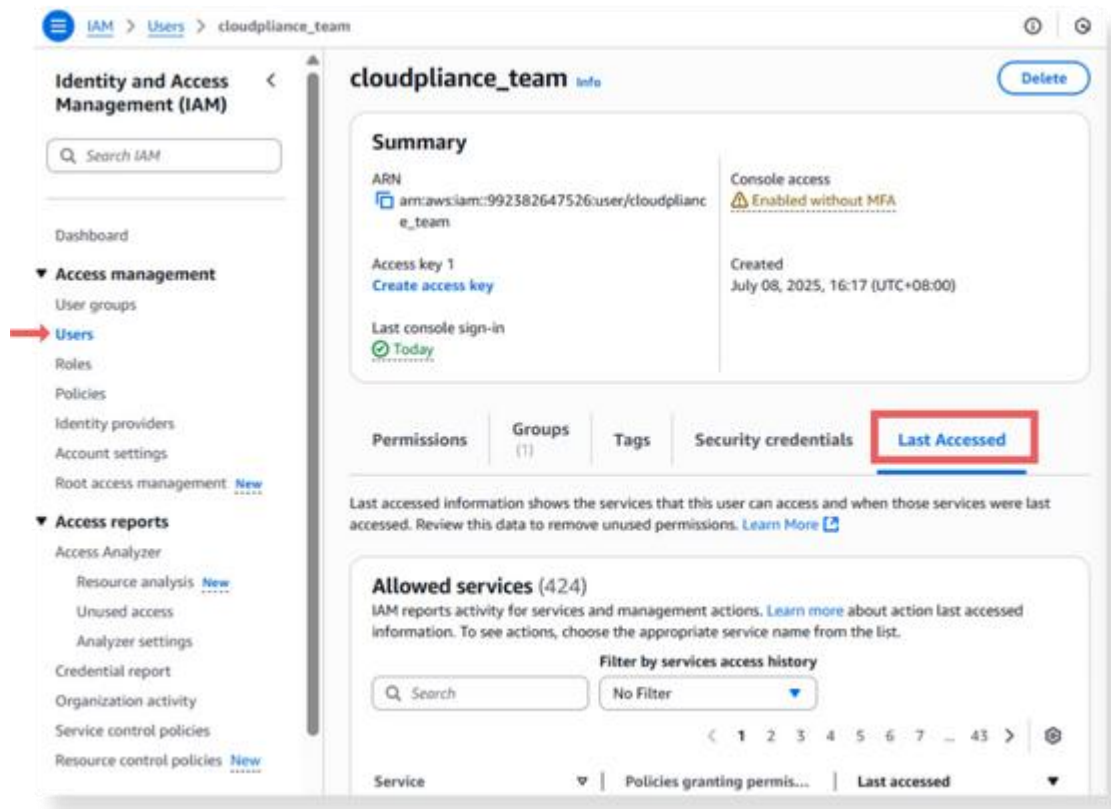


Role-Based Access Control (RBAC) means you give permissions to a role, not to every single user. Then you just assign the role as needed.

For example, you might have roles for admins, devs, or even billing folks. That way, nobody gets more access than they need, and it's easy to see who can do what.

Roles can be temporary too — someone can “assume” a role for a specific task. Pair up your roles with policies that lay out the allowed actions and resources. It keeps things tidy and secure.

Monitoring IAM Activities



Keeping an eye on IAM activities is honestly one of those things you can't skip if you care about security. It's like checking your bank account every so often — better safe than sorry.

Fire up AWS CloudTrail to log every IAM move, whether that's someone signing in, tweaking a policy, or jumping into a new role. It's basically your security camera for AWS actions.

Take some time to look through those logs. And yeah, set up alerts for weird stuff, like logins from a country you've never visited or sudden changes in permissions.

If you want to level up, try mixing CloudTrail with AWS Config or Amazon GuardDuty. These tools work together to catch threats automatically, so you can jump on anything suspicious right away.